

ALGORITHMIC OPTIMIZATION & PROBLEM-SOLVING REPORT

Name: M SAMPRIYAN

SRN: R25EJ060

Branch: CSIT 'A' REVA UNIVERSITY

Subject: Portfolio Building - Hacker Rank Solutions Analysis

CREDENTIALS & REPOSITORY LINKS

- **GitHub Repository:**

<https://github.com/SampriyanR25EJ060/Portfolio-Building/tree/main>

- **Hacker Rank Profile:**

<https://www.hackerrank.com/profile/sampriyan1327>

SUMMARY TABLE: COMPLEXITY ANALYSIS

#	Problem	Time Complexity	Space Complexity	Primary Approach
1	Diagonal Difference	$O(N)$	$O(1)$	Single-pass linear accumulation over primary and secondary diagonals
2	Dynamic Array	$O(Q)$	$O(N + Q)$	2D dynamic pointers with geometrically doubling capacity (realloc)
3	Time Conversion	$O(1)$	$O(1)$	In-place string parsing (sscanf) and conditional arithmetic formatting

#	Problem	Time Complexity	Space Complexity	Primary Approach
4	Compare the Triplets	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Single-pass element comparison across fixed 3-element arrays
5	Sparse Arrays	$\mathcal{O}(Q \cdot N \cdot L)$	$\mathcal{O}(N \cdot L)$	Sequential string comparisons using strcmp per query

PROBLEM-SPECIFIC TECHNICAL BREAKDOWN

Problem 1: Diagonal Difference

- **Optimization Mechanism:** Eliminates two-dimensional nested iteration ($\mathcal{O}(N^2)$) by recognizing that the primary diagonal element at row i is at column i , while the secondary diagonal element is at column $N - 1 - i$.
- **Implementation Strategy:** Accumulates `primary_sum += arr[i][i]` and `secondary_sum += arr[i][n - 1 - i]` in a single loop running from 0 to $N - 1$, maintaining $\mathcal{O}(1)$ auxiliary space and $\mathcal{O}(N)$ execution time.

Problem 2: Dynamic Array

- **Optimization Mechanism:** Prevents continuous static memory allocation overhead for variable-length sequence buffers using geometric array expansion.
- **Implementation Strategy:** Dynamically allocates N pointer streams (`int** arr`). When a sequence reaches capacity, its allocated memory is doubled using `realloc()`, guaranteeing an amortized $\mathcal{O}(1)$ insertion time per query across Q operations.

Problem 3: Time Conversion

- **Optimization Mechanism:** Avoids heavy string-splitting routines by taking advantage of low-level string format specifiers.
- **Implementation Strategy:** Uses `sscanf` to parse hour, minute, second, and AM/PM tokens in a single execution step. Arithmetic adjustments adjust 12 AM to

00 and 1 PM–11 PM by adding 12, executing in deterministic $\mathcal{O}(1)$ time and space.

Problem 4: Compare the Triplets

- **Optimization Mechanism:** Minimizes branch mispredictions and memory overhead for fixed-size evaluation sequences.
- **Implementation Strategy:** Evaluates $a[i]$ vs $b[i]$ for $i \in [0, 2]$. Points are updated conditionally without requiring dynamic allocation or multi-pass iterations, bounding the algorithm to $\mathcal{O}(1)$ time complexity.

Problem 5: Sparse Arrays

- **Optimization Mechanism:** Maintains contiguous array structures in memory for linear string comparisons.
- **Implementation Strategy:** Reads N input strings into a 2D char array buffer. For each of the Q queries, `strcmp` scans through the N strings. This sequential memory access pattern minimizes cache misses during linear searches.

REFLECTIVE SUMMARY ON ALGORITHMIC OPTIMIZATION

Solving these core problems in C reinforces how low-level memory management and mathematical indexing directly drive algorithm performance.

A key takeaway from the **Diagonal Difference** and **Time Conversion** tasks is the power of mathematical index reduction. Bypassing nested loops in favor of direct formulaic lookups reduces operations from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, demonstrating that structural awareness often eliminates the need for complex computation.

The **Dynamic Array** challenge highlights the crucial role of amortized analysis. Allocating memory dynamically on the heap requires balancing frequent reallocations against memory consumption. Implementing exponential capacity scaling (`capacity *= 2`) turns costly $\mathcal{O}(N)$ memory resizes into an amortized $\mathcal{O}(1)$ operation, a principle foundational to modern `std::vector` and `ArrayList` implementations.

Finally, low-level string operations in **Sparse Arrays** demonstrate the trade-offs between simple linear scanning and complex data structures like Hash Tables or Tries. While a linear scan yields $\mathcal{O}(Q \cdot N \cdot L)$ time, its implementation incurs zero memory overhead and avoids complex pointer-chasing, making it ideal for smaller datasets.

Mastering these core paradigms—index math, dynamic heap allocation, and spatial memory locality—provides a strong foundation for tackling complex system-level software engineering and algorithm design.